

MAGIK

Multimodal Agentic RAG Integrated Knowledge Assistant

A finance-domain retrieval-augmented generation system that ingests text, PDF, DOCX, XLSX, images, audio and video; routes each query through an agentic controller; retrieves with a hybrid lexical and dense pipeline; verifies its own answers against the retrieved evidence before they reach the user; and runs entirely on self-hosted open-weight models — with no third-party language-model API anywhere in the request path.

MODALITIES	MODELS COMPOSED	CI MERGE GATES	PAID LLM APIS
7 pipelines, fully isolated	18 open-weight ~42 GB · 0 fine-tuned	3 enforced retrieval · hallucination · finance	none

Contents

01	Overview	08	Guardrails
02	Architecture	09	Security & access control
03	Component models	10	Reproducibility
04	Intended use	11	Limitations & known issues
05	Training & provenance	12	Deployment
06	Evaluation	13	Responsible AI considerations
07	Answer verification	14	Provenance & references

How to read this card. Every figure here is drawn from the repository itself — the model manifest, the evaluation threshold file, the deployment configuration and the test tree — rather than from summary documentation. Figures that could not be verified were left out rather than estimated, and where the repository's own documentation disagreed with its code, the code was treated as authoritative. Open defects and a currently-failing quality gate are included in Section 06 and Section 11 rather than omitted.

01 Overview

MAGIK is built around one premise: most retrieval-augmented generation demonstrations fall apart under the conditions real traffic creates — malformed documents, adversarial input, numbers silently rounded in transit, one user's session bleeding into another's context. This system is an attempt to hold up under those conditions, and to show it with a measured evaluation harness rather than a claim.

It specialises in **financial documents** — earnings calls, 10-K and 10-Q filings, spreadsheets of financial ratios, investor presentations — because finance is an unforgiving domain for retrieval-augmented generation: a hallucinated number is worse than a hallucinated sentence. The system checks financial figures in a generated answer against the literal text of the retrieved chunks, and gates continuous integration on that check.

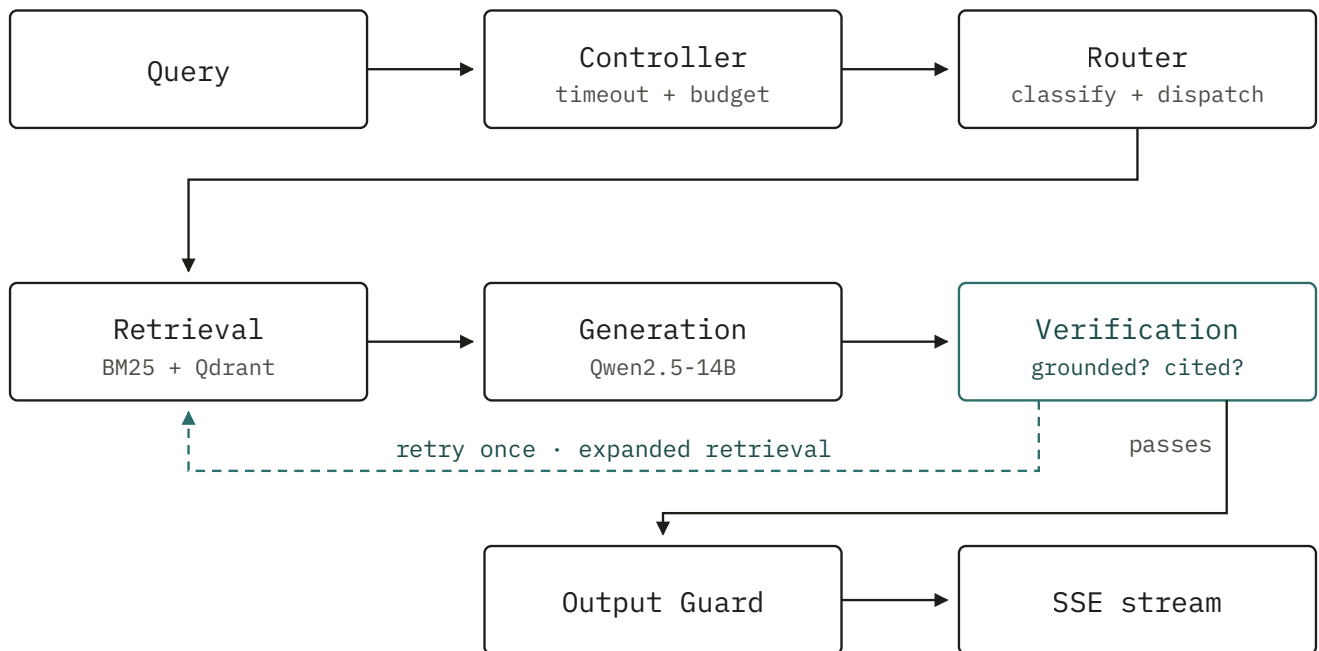
Every model in the stack is open-weight and self-hosted: a quantised GGUF language model served through `llama.cpp`, open embedding and reranking models, and open speech, OCR and vision models. There is no commercial inference API in the request path — a constraint that shapes the quantisation, latency and deployment decisions described throughout this card.

MODALITIES INGESTED	MODELS COMPOSED	PAID LLM APIS	TEST FILES
7 – text, PDF, DOCX, XLSX, image, audio, video	18 · ~42 GB on disk, all open-weight	none	142 across 8 categories

02 Architecture

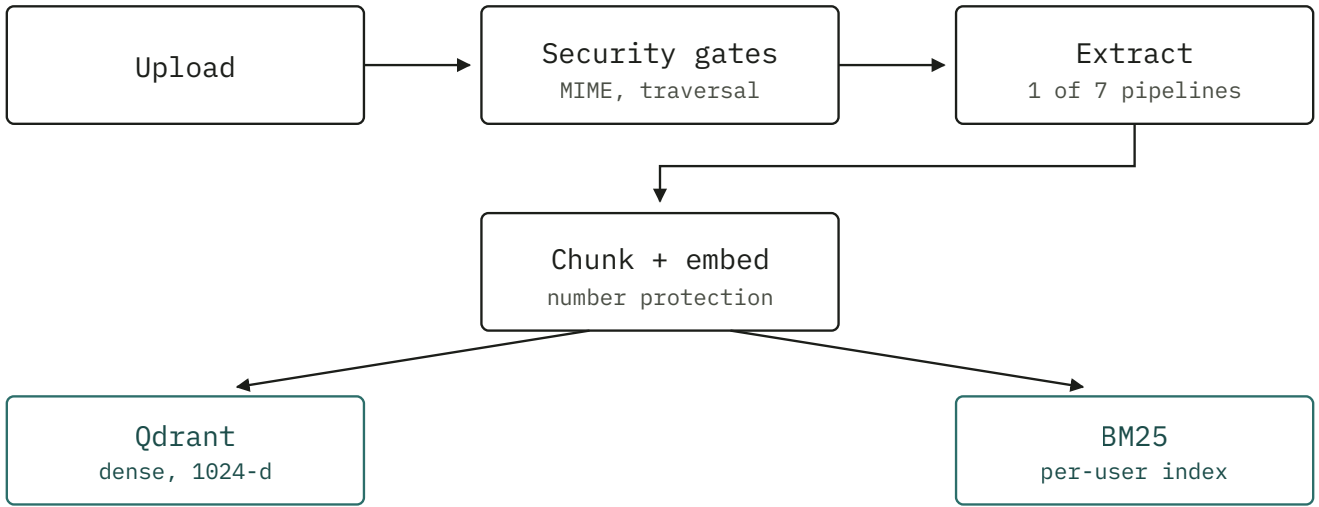
Two independent flows: a query path that answers a question, and an ingestion path that indexes a document. The query path runs under an enforced wall-clock timeout and a pre-call token budget; the ingestion path runs security gates before any file content is trusted.

Query flow



The router classifies each query once and dispatches a single tool call — a bounded dispatch, not an open-ended agent loop. A failed groundedness or citation check sends the query back through retrieval once with an expanded strategy, bounded by a hard timeout. Measured retry behaviour is reported in Section 07.

Ingestion flow



Every ingested file is written to both stores, so hybrid retrieval always has a dense and a lexical index to fuse. Audio and video ingestion return a job identifier immediately and run as a background task the client polls for progress.

Per-modality isolation

The defining structural decision: each of the seven modalities owns exactly four files — one per processing layer (ingestion, chunking, embedding, lexical indexing) — with no shared per-modality state. A defect in the spreadsheet pipeline cannot break the audio pipeline. This is also what makes the guardrail coverage claim in Section 08 countable: seven modalities across four layers gives 28 text surfaces, each of which must call the single sanitisation entry point.

Layer responsibilities

LAYER	RESPONSIBILITY
Frontend	React 19 + Vite + Tailwind single-page application
API	FastAPI routers — <code>/rag</code> , <code>/auth</code> , <code>/admin</code>
Agents	Query classification, tool dispatch, bounded execution
Retrieval	BM25 + Qdrant fusion, cross-encoder rerank, MMR diversification
Verification	Groundedness, citation accuracy, completeness, confidence, bounded retries
Guardrails	Input sanitisation, output checks, PII / SSRF / jailbreak defences, audit logging
Memory	Redis short-term + MongoDB persistent + automatic summarisation
LLM	GGUF inference in a separate <code>llama-server</code> process
Auth	JWT + Argon2 + OAuth + TOTP MFA, enforced at every data-layer boundary
Observability	structlog, Prometheus, OpenTelemetry, Grafana / Loki / Tempo
Infrastructure	Terraform, Docker, AWS EC2 GPU, Lambda wake gateway, Caddy HTTPS

03 Component models

MAGIK is a compound system rather than a single trained model. Eighteen open-weight checkpoints — roughly 42 GB on disk — are composed through the pipeline above. Each is pinned to an exact upstream commit hash rather than a moving default branch, and the provisioning manifest records a SHA-256 on first download and re-verifies it on every subsequent run, so a corrupted or silently swapped upstream file is caught rather than served.

Provisioned model manifest · all open-weight, none fine-tuned

ROLE	CHECKPOINT	SIZE
Generation (LLM)	Qwen2.5-14B-Instruct · Q4_K_M GGUF	9.00 GB
Vision — charts & images	Qwen/Qwen2-VL-7B-Instruct	16.59 GB
Vision — video frames	Qwen/Qwen2-VL-2B-Instruct	2.20 GB
Evaluation judge	Qwen2.5-7B-Instruct · Q4_K_M GGUF	4.70 GB
Cross-modal embedding	google/siglip-so400m-patch14-384 · 1152-d	1.76 GB
Speech recognition	Systran/faster-whisper-large-v3	1.55 GB
Text embedding	BAAI/bge-large-en-v1.5 · 1024-d	1.35 GB
Reranking	BAAI/bge-reranker-large · cross-encoder	1.34 GB
Image captioning	Salesforce/blip-image-captioning-large	0.90 GB
Speaker diarization	pyannote/speaker-diarization-3.1	0.60 GB
Financial sentiment	yiyanghkust/finbert-tone	0.44 GB
Named-entity recognition	dslim/bert-base-NER	0.43 GB
Toxicity screening	Detoxify (original)	0.42 GB
OCR — printed text	microsoft/trocr-large-printed	0.36 GB
Diarization — segmentation	pyannote/segmentation-3.0	0.20 GB
Diarization — embedding	pyannote/wespeaker-voxceleb-resnet34-LM	0.10 GB
Keyword extraction	all-MiniLM-L6-v2 (KeyBERT)	0.09 GB
OCR — scene text	EasyOCR (en)	0.06 GB

The evaluation judge is never loaded during request serving — it runs only when the harness grades a run, and it consolidated three previously separate judging paths onto a single model so that scores produced by different tools are comparable.

04 Intended use

In scope

- Question-answering over finance documents a user has uploaded — filings, earnings-call audio and video, spreadsheets, presentations, chart images.
- Demonstrating production-oriented RAG engineering: hybrid retrieval, answer verification, guardrail coverage, tenant isolation, observability and CI-gated evaluation.
- Single-GPU, low-concurrency deployment.

Out of scope

- Not financial advice. It answers questions about documents; it does not recommend financial decisions, and answers should be checked against the cited source before being acted on.
- Not evaluated on non-English documents.
- Not load-tested at commercial multi-tenant scale.
- The public demonstration runs a deliberately open shared account — not a place to store confidential documents.

05 Training & provenance

No model in this system has been fine-tuned. All eighteen checkpoints run at their published open weights — no LoRA, no PEFT, no custom training run anywhere in the stack. That is a deliberate scope choice, stated plainly rather than left ambiguous: the engineering investment here goes into retrieval quality, answer verification, guardrail coverage and evaluation rigour around off-the-shelf models. Training models and engineering retrieval systems are different skills, and this project is scoped around the second.

Where a well-known library is used narrowly, that is said rather than implied. Text splitting uses `langchain-core` and `langchain-text-splitters` only — the agent's tool routing is a hand-built typed dispatch, not a LangChain agent or chain.

06 Evaluation

Eleven suites are runnable from the harness command line — retrieval, generation, hallucination, behavioral, OCR, audio, video, routing, end-to-end, multimodal and regression. Three are enforced as hard merge gates: **retrieval**, **hallucination** and **finance numeric fidelity**. Everything else is measured and reported as informational rather than presented with false parity.

Measurement methodology, learned the hard way. Three back-to-back runs on identical code once produced faithfulness scores of 0.363, 0.592 and 0.589. The outlier was not judge stochasticity — both generation and judging run greedily at temperature zero — it was the first run after a server restart, with embedding, key-value cache and judge-subprocess warm-up costs landing on it. Every quality metric read 15 to 20 points worse, with visibly higher latency alongside.

The resulting process rule: discard the first run after any restart, and average at least three. Baselines labelled *N=3-averaged* below follow it. Where a later confirmation run surfaced a genuinely worse value than the average, the *gate is set from the observed maximum rather than the average* — a regression gate has to reflect behaviour the system actually exhibits, not the behaviour it exhibits on a good day.

Retrieval — enforced gate

Baseline v5, measured on the production GPU box against a 56-query gold set re-aligned to that box's own chunking. Each floor sits at 95% of baseline, so a 5% regression fails the merge.

retrieval suite · n = 56 · **CI-gated** · latest column is the most recent staging run

METRIC	V5 BASELINE	CI FLOOR	LATEST	STATUS
recall@5	0.5089	0.4835	0.4464	breach
MRR	0.3558	0.3380	0.3069	breach
nDCG@10	0.4024	0.3823	0.3642	breach
recall@10	0.5536	0.5259	0.5625	pass
hit rate	0.6786	0.6447	0.8036	pass
context precision	0.0268	0.0255	0.0321	pass

OPEN – GATE LEFT RED, DELIBERATELY NOT RE-BASELINED

Three of six gated retrieval metrics are breached while three improved. That pattern does not fit a "quality got worse" story: **coverage improved while ordering degraded** — the right chunks are found more often than before, but ranked lower within the result set.

Two candidate causes are under attribution, and they need different fixes. A metadata-backfill step added to result fusion is documented as ranking-neutral but is not: newly populated fields change both a filter outcome and a modality score boost that multiplies matching chunks by 1.5 to 2.5 times, which reorders the top five in exactly this signature. Alternatively the comparison may not be like-for-like, since the baseline was measured on the production box and this run on the private staging box — the pre-deploy quality gate described in Section 12 — against a snapshot-derived lexical index.

The gate stays red until the cause is attributed, rather than being re-baselined to the new numbers. Re-baselining now would encode whichever cause it is as "expected" — precisely the outcome the gate exists to prevent. It appears on this card because a system card listing only passing numbers tells a technical reader less than one showing how regressions are actually handled.

Hallucination — enforced gate

Baseline v8, measured across **all seven modalities** on 97 answered gold rows, averaged over three consecutive warm runs. This suite was informational at the previous release and is now gate-enforced.

hallucination suite · n = 97, all 7 modalities · N=3-averaged · **CI-gated**

METRIC	V7	V8	GATE	MEANING
fabrication rate	0.0653	0.0619	max 0.079	Primary safety signal — ungrounded numbers, template leakage
hallucination rate	0.2715	0.2234	max 0.246	Blended metric, kept for continuity
omission rate	0.2302	0.1822	—	Completeness signal, not fabrication — no gate

The fabrication gate is set at 10% above the *observed maximum* across four runs (0.0722), not the three-run average (0.0619). A fourth confirmation run surfaced a real, intermittent failure that three identical runs had missed: on one audio query the model conflated an unrelated inflation projection from elsewhere in the same transcript into a job-vacancy answer. That was root-caused against the raw transcript and confirmed as a genuine model hallucination rather than a metric artifact — so the gate reflects it instead of averaging it away.

Finance numeric fidelity — enforced gate

Financial figures cited in an answer are matched against the literal text of the retrieved chunks within a 0.5% tolerance and *no unit-scale bridging* — "1.2 billion" is not silently accepted as support for "1,200". The gate requires at least 95% of cited figures to be traceable to retrieved context. This is the guarantee the entire finance-domain framing rests on, so it is enforced rather than merely measured.

Generation and routing — informational

These route through a language-model judge against a live server, which makes re-baselining a GPU-box exercise. The figures below are the honest three-run average over the default generation corpus (text, PDF and DOCX; 42 rows) and are not gate-enforced.

generation suite · n = 42 · N=3-averaged · informational

METRIC	VALUE	METRIC	VALUE
Answer correctness	0.7083	Answer relevancy	0.6528
Context recall	0.8962	Faithfulness	0.5146
Finance fidelity	0.8266	Route accuracy	1.000

07 Answer verification

Before an answer reaches the user, a verification loop checks whether its claims are supported by the retrieved context and whether its citations point at real chunks, retrying once with an expanded retrieval strategy when a check fails. At the previous release these metrics existed in the request path but had never been scored. They are now measured.

GROUNDING SUCCESS 0.9384	CITATION ACCURACY 0.8587	VERIFICATION LATENCY p50 2.44 s p95 6.81 s	MEAN RETRIES / QUERY 0.2898
-----------------------------	-----------------------------	--	--------------------------------

V7 TO V8 – WHAT WIRING THE LOOP PROPERLY ACTUALLY BOUGHT

Grounding success rose from 0.8587 to 0.9384. Mean retries per query fell from 0.50 to 0.29 while retry success rose from 0.071 to 0.186 — fewer retries, each more useful. Verification latency roughly halved: p50 from 4.42 s to 2.44 s, p95 from 11.07 s to 6.81 s.

Retry effectiveness is reported honestly rather than as a headline. Across 140 retried sessions, **88.6% changed nothing, 8.6% raised the score, and 1.4% flipped a failing answer to a passing one**. The loop is a safety net that rarely fires usefully, not a general quality multiplier — and stating that is more useful to an engineer evaluating the design than quoting the 1.4% on its own.

A bug this instrumentation caught. Building a streaming client so the harness could finally exercise the endpoint the user interface actually calls surfaced a defect invisible to every previous run: plain-text documents never triggered the verification loop at all. The ingestion path tagged chunks "text" while the chunking path used the canonical "txt", so a modality check silently evaluated false for every text query. It was fixed by aliasing the two tags, which repaired gating for all already-indexed chunks without re-ingestion.

08 Guardrails

All ingested content — documents, transcripts, web-search results — is treated as untrusted data, never as instructions. A single entry point is called on every one of the 28 modality-by-layer text surfaces before that text reaches a model, so coverage is a countable property rather than a matter of reviewer diligence.

ATTACK RECALL 64 / 64 (100%)	FALSE POSITIVES 0.9% · F1 0.994	DETECTION PATTERNS 43, severity-tiered	OWASP LLM TOP 10 10 / 10 addressed
---------------------------------	------------------------------------	---	---------------------------------------

Those results are measured against a 109-case red-team corpus spanning prompt injection, jailbreak, encoding bypass, PII exposure, server-side request forgery and poisoned-document attacks. The repository carries 306

guardrail test cases over the same surfaces. Every model response is checked before it reaches the client, including a fallback notice path for web-sourced answers that bypass the verification loop, and every guardrail violation is written to a persistent audit log rather than silently dropped.

09 Security & access control

Authentication is a full implementation rather than a demonstration login: JWT access and refresh tokens (HS256), Argon2 password hashing with a bcrypt fallback, Google OAuth 2.0 authorization-code sign-in with CSRF state validation, TOTP multi-factor authentication, a Redis-backed token blacklist so that logout and revocation actually take effect, and per-user request rate limiting on a 60-second window. Production secrets live in AWS Parameter Store as encrypted values, are fetched fresh at deploy time, written to a mode-0600 file, and never committed.

Tenant isolation

Every data layer filters on the user identifier independently. There is no layer where isolation is delegated upstream to another — so no single missed filter can expose one tenant's data to another.

DATA LAYER	ISOLATION MECHANISM
Qdrant (vectors)	Typed field condition on <code>user_id</code> in every query filter
BM25 (lexical)	Per-user index file path — no shared index exists
Redis (memory, cache)	Namespaced keys under <code>user:{user_id}:</code>
MongoDB (history)	Every query filters on <code>user_id</code> explicitly

10 Reproducibility

A system whose evaluation numbers cannot be reproduced is a system whose evaluation numbers do not mean very much. Four things are pinned so that a box built today builds the same system that was measured:

- **Model weights** — every checkpoint pinned to an exact upstream commit hash rather than a moving branch, with SHA-256 verification on every provisioning run.
- **Python dependencies** — fully locked in a committed lockfile.
- **Infrastructure** — codified in Terraform rather than configured by hand.
- **Vector data** — Qdrant collections can be snapshotted and restored on demand.

This mattered concretely: turning on strict manifest checking for the first time revealed that the seven-billion-parameter vision model had been required at start-up all along but was never downloaded or checksum-verified by the provisioning manifest — silently absent, and invisible until the pin was enforced.

11 Limitations & known issues

Documented deliberately rather than left implicit. A system that lists only its strengths is less credible, not more.

- **Three retrieval gate metrics are breached and unattributed** — see Section 06. Left red on purpose rather than re-baselined.
- **Retrieval context precision is low in absolute terms** (0.027 at baseline). It is gated against further drift, but the absolute value needs a dedicated retrieval-quality pass — coarser chunking, better reranking — which has not been scheduled.

- **A real, intermittent hallucination remains open.** On dense audio transcripts the model occasionally conflates two unrelated numeric figures from the same document. It is root-caused, low-frequency, reflected in the gate, and not yet fixed.
- **The hybrid web route does not execute a live web search** on that path — a known open defect, thresholded at zero so it cannot silently appear to pass.
- **The default generation suite covers only text, PDF and DOCX.** Image and spreadsheet rows are excluded from the default run and must be requested explicitly, so headline generation numbers under-represent those modalities.
- **Streaming evaluation is manual, not continuous.** The harness can now exercise the endpoint the interface actually calls, but on a nightly or on-demand basis; a full-corpus automated run is not yet wired into the gate.
- **A modality-tagging audit is incomplete.** The tag mismatch fixed for text was found by inspection; the PDF, DOCX and spreadsheet ingestion paths construct tags inline in the same way and have not been fully audited.
- **Finance numeric fidelity is enforced at merge time, not sampled from live traffic.**
- **No formal fairness or bias audit has been conducted.** The system reasons over financial documents rather than making decisions about people, but that is a scope argument, not an evaluation.
- **Single production instance with no horizontal scaling or failover.** The second GPU instance described in Section 12 is a pre-deploy quality gate, not a hot standby — a production outage has no automatic failover target. The cost model is the reason, and it is a real availability limit.

12 Deployment

Production runs on a single AWS GPU instance — an NVIDIA L40S with 48 GB of video memory — behind a wake gateway and a scheduled idle-stop function. The box stops itself after 20 minutes without traffic, subject to a minimum-uptime guard so that it cannot kill an instance which is still warming up, mid-deploy, or running an evaluation job.

As of this release the gateway starts the GPU **only on an explicit human click**, never on an automated poll. Uptime monitors, crawlers and health probes can hit the entry point as often as they like without costing anything.

ALWAYS-ON COST ~\$1,340 / month	SCALE-TO-ZERO COST ~\$12 / month fixed + a few \$ / active hour	COLD START 60-90 s, first request	TRANSPORT HTTPS via Caddy; app port never public
---	--	---	---

That is the whole trade: roughly a hundredfold reduction in infrastructure cost, paid for with a minute of latency on the first visit after an idle period. For a portfolio system with bursty, demonstration-driven traffic it is the right side of the trade; for continuous production traffic it would not be.

Production vs. staging

Two identically-provisioned GPU instances exist, not one — a public production box and a private staging box used solely as a pre-deploy quality gate. Staging carries no Elastic IP, no public port, no Caddy reverse proxy and no monitoring stack; the only way onto it is AWS Systems Manager, and it never serves user traffic.

INSTANCES 2 x g6e.xlarge (L40S), identical hardware	STAGING ACCESS SSM only, no public port or Elastic IP	PROMOTION GATE Full Tier-2 quality suite must pass	STAGING UPTIME Minutes per deploy, not continuous
---	---	--	---

Every tagged release follows a champion-to-successor pipeline: the production image is built exactly once, deployed to staging, and the full Tier-2 RAG-quality suite — the same suite behind the hallucination gate in Section 06 — runs

against it. Only if that gate passes does the identical image, never rebuilt or re-tagged, get promoted to production. A failing gate rolls staging back to its previous image and alerts; production is skipped entirely rather than rolled back, since nothing was ever deployed there. Staging wakes only for the few minutes a deploy takes and is stopped immediately afterward, so it does not double the always-on cost figure above. Both boxes share the same external Qdrant, Redis and MongoDB services, isolated by tenant scoping rather than fully separate infrastructure — a documented tradeoff, not an oversight.

A bug this separation caught. Every structured log line carries an `env` field, and staging was found emitting `"env": "production"` on all of them — the deploy step layered an environment file that never defined `ENV` on top of an AMI clone that already had it set, so production's value silently survived. Left unfixed, staging and production traffic would have been indistinguishable in the shared Grafana and Loki dashboards.

Observability on production is a full stack rather than a health endpoint: structured JSON logs aggregated in Loki under a shared trace identifier, Prometheus counters on every per-modality class scraped on a dedicated port, OpenTelemetry spans exported to Tempo, Grafana dashboards behind reverse-proxy authentication, and alerting on circuit-breaker trips, ingestion error spikes, latency breaches and hallucination-rate drift. Continuous integration runs lint, type checks and unit tests on every push, and a retrieval regression gate on every pull request, ahead of the staging-to-gate-to-promote pipeline above.

13 Responsible AI considerations

- **No external inference provider.** Because every model is self-hosted, document and query content never leaves the deployment boundary for a third-party language-model API. This is the single largest privacy property of the design.
- **PII detection** is integrated across ingestion and memory surfaces using Microsoft Presidio.
- **Toxicity screening** runs on model output through Detoxify.
- **Untrusted-content discipline.** Ingested documents, transcripts and web results are always handled as data, never as instructions — the principle the guardrail design in Section 08 implements.
- **Auditability.** Every guardrail violation is written to a persistent audit log, so a blocked request leaves a reviewable trace.
- **Answer traceability.** Answers carry citations to retrieved chunks, and the finance-fidelity check exists so that a cited number can be traced back to source text rather than trusted on the model's word.

14 Provenance & references

Live demonstration	magik.vk-ai.online — the GPU box wakes on your click; the first response takes 60 to 90 seconds
Repository	github.com/vjkarthik98/MULTIMODAL-AGENTIC-RAG-INTEGRATED-KNOWLEDGE-AI-ASSISTANT
Release	v1.0.1 · 30 August 2026
License	MIT
Author	Vijaya Karthik
Engineering log	Full change history is maintained in-repo, including root-caused production incidents rather than only successful changes

Every figure in this card was read from the repository at release v1.0.1 — the model manifest, the evaluation threshold file, the deployment configuration and the test tree. Figures that could not be verified were omitted rather than estimated. Where the repository's own documentation disagreed with its code, the code was treated as authoritative.